

Design note — ROS 2 Planar Manipulator

This project controls a simulated three-joint planar arm and shows its movement in a PyQt5/PyQtGraph GUI. It keeps the given link lengths, joint limits and ground constraint. The supplied `planar_arm.py` is unchanged. The 3D view adds a clearer picture of the same planar arm; it does not change the robot's geometry.

The controller owns the motion and pick-and-place sequence. The GUI sends requests and displays feedback, so closing it does not stop the controller. Planning and execution are separate Python components inside the controller. An execution backend connects the planner to software simulation or Gazebo. This keeps device-specific commands out of the planner and leaves a clear place for a future hardware driver. A separate recorder saves telemetry without putting file writes inside the control loop.

For each target, the planner checks the input, calls the supplied inverse kinematics, and checks the returned angles and final position. It also checks the direct path between the starting and finishing poses, because valid endpoints alone do not prevent a link crossing the ground. Uncertain or invalid paths are rejected. A quintic trajectory gives a smooth start and stop, and its duration is increased when needed to meet the chosen reference speed and acceleration limits. This is a direct-path planner, not a general obstacle planner.

The controller publishes joint feedback and references at a target rate of 50 Hz, with task status at 10 Hz. `/joint_states` reports the backend state; `/joint_commands` reports the desired trajectory and is not a command input. The GUI matches their timestamps before plotting tracking error. Qt updates widgets on its main thread, while ROS callbacks run in a worker thread. Service calls are asynchronous, and stale telemetry disables new GUI commands.

Pick-and-place runs through approach, pick, lift, transfer, place and retreat. All motion legs are checked before the task starts. Each phase waits for the required motion or grasp/release condition rather than assuming a fixed delay means success. A service response confirms acceptance; status with the same command ID reports the later result. Busy requests are rejected. Cancel requests a hold and preserves an already-held object. Software reset is not hardware homing.

Position mode provides ideal playback. Velocity/PID mode adds actuator lag and error correction. Gazebo adds physical simulation, but grasping uses an attachment constraint rather than frictional contact; the object settles about 5 mm below its release height. Validation includes 22 unit/regression tests, 25 repeat and recovery checks, an independent ROS observer, and fresh-workspace build checks. The repository includes recordings, results and attribution for the reused logging helper.

With more time, the next steps would be a standard ROS action, moving every motion-leg replan outside the timer's shared lock, broader collision checking, and a hardware driver with measured feedback and a device-side watchdog. The current Python system does not guarantee hard real-time timing or hardware safety.